

CHAPTER-2

Types of Variables in Java :

Variables in Java are named memory locations used to store data. Java has three main types of variables: **Local Variables**, **Instance Variables**, and **Static Variables**. A **Local Variable** is declared inside a method, constructor, or block and is accessible only within that scope. An **Instance Variable** is declared inside a class but outside methods, and each object has its own copy. A **Static Variable** is declared with the static keyword and is shared among all objects of the class. These variables help store and manage data efficiently during program execution.

TYPE CASTING

Type casting in Java is the process of converting one data type into another. It is useful when performing operations between different data types or when assigning a larger data type to a smaller one. Java supports two types of type casting: Implicit (Widening) and Explicit (Narrowing). Implicit casting is done automatically by the Java compiler when converting a smaller data type to a larger one. Explicit casting is done manually by the programmer using the cast operator because converting a larger data type to a smaller one may result in data loss.

Types of Type Casting

1. Implicit Type Casting (Widening)

- Done automatically by Java.
- Converts a smaller data type to a larger data type.
- No data loss occurs.
- Example: int → long → float → double

2. Explicit Type Casting (Narrowing)

- Done manually using the cast operator ().

- Converts a larger data type to a smaller data type.
- Data loss may occur.
- Example: double → float → long → int → short → byte

Example

// Implicit Type Casting

```
int a = 100;
```

```
double b = a;
```

```
System.out.println(b); // 100.0
```

// Explicit Type Casting

```
double x = 99.99;
```

```
int y = (int) x;
```

```
System.out.println(y); // 99
```

Why Do We Need Type Casting in Java?

Type casting is used in Java to **convert one data type into another** so that different data types can work together correctly. It helps avoid type mismatch errors and allows programmers to control how data is stored and processed.

Reasons to Use Type Casting

1. **To convert one data type to another.**
2. **To avoid type mismatch errors** during assignments or calculations.
3. **To perform operations between different data types.**
4. **To store a larger data type in a smaller data type** (using explicit casting).

5. **To save memory** when a smaller data type is sufficient.
6. **To use methods or APIs** that require a specific data type.
7. **To control data conversion** according to program requirements.

Example

```
double price = 199.99;
```

```
int amount = (int) price; // Explicit type casting
```

```
System.out.println(amount); // Output: 199
```

Here, the double value is converted to an int. The decimal part (.99) is removed.

Type Casting with Characters (char) in Java:

In Java, the char data type stores a **single Unicode character**. Since each character has a numeric Unicode (ASCII for common English characters) value, char can be type cast to and from numeric data types.

Character Type Casting Rules

1. A char can be **implicitly converted** to int, long, float, or double.
2. An int (or other numeric type) can be **explicitly converted** to char using the cast operator (char).
3. char cannot be converted to boolean or vice versa.
4. The Unicode value of the character is used during conversion.

Example 1: char to int (Implicit)

```
char ch = 'A';
```

```
int num = ch;
```

```
System.out.println(num);
```

Output:

65

Example 2: int to char (Explicit)

```
int num = 66;
```

```
char ch = (char) num;
```

```
System.out.println(ch);
```

Output:

B

Example 3: char to double

```
char ch = 'C';
```

```
double d = ch;
```

```
System.out.println(d);
```

Output:

67.0

Common Character Values

Character Unicode/ASCII Value

'A'	65
'B'	66

Character Unicode/ASCII Value

'C'	67
'a'	97
'b'	98
'0'	48
'1'	49

Operators in Java

Operators are special symbols used to perform operations on variables and values.

Types of Operators in Java

1. Arithmetic Operators
2. Assignment Operators
3. Relational (Comparison) Operators
4. Logical Operators
5. Unary Operators
6. Bitwise Operators
7. Shift Operators
8. Ternary Operator

1. Arithmetic Operators

Operators: +, -, *, /, %

Example Program

```
class ArithmeticDemo {  
    public static void main(String[] args) {  
        int a = 20, b = 10;  
  
        System.out.println("Addition = " + (a + b));  
        System.out.println("Subtraction = " + (a - b));  
        System.out.println("Multiplication = " + (a * b));  
        System.out.println("Division = " + (a / b));  
        System.out.println("Modulus = " + (a % b));  
    }  
}
```

2. Assignment Operators

Operators: =, +=, -=, *=, /=, %=

Example Program

```
class AssignmentDemo {  
    public static void main(String[] args) {  
        int a = 10;  
  
        a += 5;  
  
        System.out.println(a);  
    }  
}
```

```
a -= 3;

System.out.println(a);


a *= 2;

System.out.println(a);
}
}
```

3. Relational Operators

Operators: ==, !=, >, <, >=, <=

Example Program

```
class RelationalDemo {

    public static void main(String[] args) {

        int a = 20, b = 15;


        System.out.println(a > b);
        System.out.println(a < b);
        System.out.println(a == b);
        System.out.println(a != b);
    }
}
```

4. Logical Operators

Operators: &&, ||, !

Example Program

```
class LogicalDemo {  
    public static void main(String[] args) {  
        int age = 20;  
  
        System.out.println(age > 18 && age < 30);  
        System.out.println(age < 18 || age == 20);  
        System.out.println(!(age > 18));  
    }  
}
```

5. Unary Operators

Operators: +, -, ++, --, !

Example Program

```
class UnaryDemo {  
    public static void main(String[] args) {  
        int a = 10;  
  
        System.out.println(++a);  
        System.out.println(a++);  
        System.out.println(a);  
        System.out.println(--a);  
        System.out.println(a--);  
    }  
}
```

6. Bitwise Operators

Operators: &, |, ^, ~

Example Program

```
class BitwiseDemo {  
    public static void main(String[] args) {  
        int a = 5, b = 3;  
  
        System.out.println(a & b);  
        System.out.println(a | b);  
        System.out.println(a ^ b);  
        System.out.println(~a);  
    }  
}
```

7. Shift Operators

Operators: <<, >>, >>>

Example Program

```
class ShiftDemo {  
    public static void main(String[] args) {  
        int a = 8;  
  
        System.out.println(a << 2);  
        System.out.println(a >> 1);  
        System.out.println(a >>> 1);  
    }  
}
```

```
}
```

8. Ternary Operator

Operator: ? :

Example Program

```
class TernaryDemo {  
    public static void main(String[] args) {  
        int a = 10, b = 20;  
  
        int max = (a > b) ? a : b;  
  
        System.out.println("Largest = " + max);  
    }  
}
```